1. Machine Learning Implementation

Use your knowledge of machine learning to answer the following questions. You could either set up some math (no need to solve) or write pseudocode, your choice (unless otherwise noted)!

A) Use Bayesian inference to calculate the relative likelihood of p(θ | data) given the following prior probabilities: p(θ=0.5) = 0.1 and p(θ=0.25) = 0.2 and data = Heads, heads, tails, tails, heads and where θ represents likelihood of heads. If you go the pseudocode route, you don't need to calculate each scenario.

Under θ=0.5: 0.5*0.5*0.5*0.5*0.5 * 0.1
Under θ=0.25: 0.25*0.25*0.75*0.75*0.25 * 0.2

B) You have a piece of code (say a function/method) called `SVM` that takes `data` and calculates a model (say, called with `x = SVM(data)`). You also have a function/method called `evaluate` that takes the output of `SVM` and returns an integer with how good the model is. Write a brief bit of pseudocode that performs leave-one-out cross validation.

```
arrayOfEvals = new array [number of elements in data];
For i=1->number of elements in data:
newData = data without element i
x = SVM(newData)
arrayOfEvals[i] = evaluate(x)
printout(median(arrayOfEvals))
```

C) You have the following data, and you'd like to learn "happy" vs "sad" based on the associated variables:
Point A: happy, +1, "cherries"
Point B: happy, -1, "cherries"
Point C: sad, -1, "blueberries"
Develop a tree (hand-drawn) or write pseudocode to build a decision tree as discussed in class (i.e. greedily) with this data:

Start (if "cherries") -> happy

Start (if "blueberries") -> sad

Not a very complicated tree!

2. Designing an Algorithm

Design a machine learning algorithm to determine if the weather is going to be "sunny," "partly sunny," "cloudy," "rainy," or "snowy" based on several variables: the predicted high temperature, the predicted low temperature, the historical state (on this day in the past, what was the state?), and the state the previous day. You won't need to implement anything, just make a few choices, as follows:

A) What algorithm would you select to address this problem? Why? What are some of the advantages or disadvantages of your selected method?
You've got a couple of options here. I'd go with a naïve Bayes classifier as this is very much a classification problem (only 5 possible value that you want to predict), and you have a good number of complicated variables. The big disadvantages of this approach is that the temperatures are close to continuous, so you'd need plenty of training data, but the independence assumption should help minimize this issue.

B) What parameter(s) would you need to select and either what values would you pick or what method would you use to pick them?
Thankfully, with a naïve Bayes classifier, I don't need to select any parameters, that's one of the advantages of this method!

C) How would you check your method for correctness of fit?
Assuming that we don't need to know tomorrow's weather immediately, I would make a series of predictions, wait until that day happens, and see if I was right or wrong. I would use this to validate my method. Note that other answers are acceptable here as well (cross validation, for example).

3. Selection

Select an algorithm for each of the following scenarios. Please explain WHY you are selecting your algorithm of choice by listing any advantages/disadvantages of your method (i.e. why the advantages are particularly advantageous or why the disadvantages are not too big of a deal).

A) You want to determine if an email is spam or not based on if there has been previous contact, what domain it is from, and the time of day that it was sent. You have a very large amount of training data.

I would use a support vector machine (SVM). This problem is a binary classification-based problem, so it would work, it is an efficient algorithm with large amounts of training data, and it can facilitate online learning if that's desired.

B) You want to predict how acidic soil will be based on global coordinates (longitude and latitude), how "developed" the area is (urban, suburban, rural, or wilderness), and the amount of annual rainfall.

I would use regression since I want to predict a continuous value. It would be slow since I would have to try many degrees of polynomial to find the best fit (and would require further validation to ensure no overfitting was happening), but this application doesn't sound urgent and wouldn't need to be online.

C) You want to predict if a consumer will want to "buy" or "not buy" a video game based on other video games that they've purchased. You have a variable amount training data for each consumer (some will have a lot of training data [i.e., they've purchased a relatively large number of games], while others will have a little). This will be based on the genre, the developer, and the operating system requirements of previous games purchased.

The difficulty of working with variable, small amounts of training data is a serious issue, but it wouldn't be too bad if you're occasionally wrong. So, I'd once again use an SVM model.

4. Fix it, Felix!

There's something wrong with the following scenarios. Explain what's wrong and fix it. No math or coding is required.

A) You are trying to classify jokes as funny or not funny based on two parameters and you use perceptron to do this because it is very quick.
You used perceptron. That's always going to be wrong because it fails if the data isn't linearly separable, which it almost never is.

B) You are using Bayesian inference to determine if a coin is magic or not. You assign prior probability that the coin lands on heads half the time, $p(\theta = 0.5)$, as 0. I mean, the coin's magic, so this makes sense, right?
You shouldn't really ever have a prior probability of absolute zero because you're then saying that it can *never* happen under your model, which is sometimes warranted, but certainly not in this case since we aren't completely positive that the coin couldn't possibly be fair. It could be magic in other ways…

C) You want to model a home's sale price based on square footage, zip code, year built, and year of most recent remodel. You get this information (as well as the final sale price) in your training data and use linear regression to fit a line. The fit is poor, but you know you have strong training data and never specified any parameters.
Don't use *linear* regression, the relationship likely isn't linear, but by using linear regression, you're forcing the model come up with the best fit that is linear (which in this case seems to be a quite poor fit). You can use polynomial regression while varying the degree of polynomial until the fit is strong, as indicated by using cross validation.